

1. Ringkasan

Sistem perlu mengirim data monitoring device IoT ke endpoint HTTP/S client sesuai jadwal. Setiap data pengiriman disimpan di database dengan status tertentu agar prosesnya bisa dilacak, diulang saat gagal, dan tidak terkirim ganda.

2. Alur Pengiriman Normal

1. Scheduler berjalan secara berkala dan mengambil data berstatus `PENDING`.
2. Data dikunci dalam transaksi lalu diubah menjadi `PROCESSING`.
3. Worker membentuk payload JSON dari data monitoring.
4. Worker mengirim payload ke endpoint HTTP/S client.
5. Jika response client adalah HTTP `2xx`, status diubah menjadi `SENT`.
6. Field `sent_at` diisi dengan waktu pengiriman berhasil.

Untuk mencegah data diproses lebih dari satu worker, pengambilan data dapat memakai lock database seperti `SELECT ... FOR UPDATE SKIP LOCKED`.

3. Flow Normal

```
[Scheduler]
  |
  v
[Ambil data PENDING]
  |
  v
[Lock sebagai PROCESSING]
  |
  v
[Kirim payload HTTP/S]
  |
  v
[Response 2xx?] -- ya --> [Status SENT + sent_at]
```

4. Mekanisme Retry

Jika pengiriman gagal karena timeout, network error, HTTP `5xx`, `408`, atau `429`, error dianggap sementara dan data boleh dicoba ulang.

Saat gagal sementara:

1. Status diubah menjadi `FAILED`.
2. `retry_count` dinaikkan satu.
3. `last_error` menyimpan ringkasan error terakhir.
4. `next_retry_at` dihitung menggunakan exponential backoff.
5. Scheduler retry mengambil data `FAILED` yang `next_retry_at`-nya sudah lewat.
6. Jika retry berhasil, status menjadi `SENT`.
7. Jika `retry_count` sudah mencapai `max_retry`, status menjadi `DEAD_LETTER`.

Error permanen seperti HTTP `400`, `401`, `403`, dan `404` tidak di-retry otomatis karena biasanya disebabkan payload salah, credential tidak valid, akses ditolak, atau endpoint client tidak ditemukan. Data seperti ini ditandai untuk investigasi manual, misalnya dengan status `DEAD_LETTER`.

5. Flow Retry

```

[Worker kirim HTTP/S]
|
v
[Gagal kirim]
|
+--> [Timeout / network / 5xx / 408 / 429]
|
|
|       v
|   [retry_count + 1, simpan last_error]
|
|       v
|   [Hitung next_retry_at]
|
|       v
|   [retry_count < max_retry?]
|
|       +-- ya --> [FAILED, tunggu retry]
|
|       +-- tidak --> [DEAD_LETTER]
|
+--> [400 / 401 / 403 / 404]
|
|
|       v
|   [DEAD_LETTER / investigasi manual]

```

6. Field Pendukung

Field	Keterangan
id	ID unik data pengiriman.
device_id	ID device sumber data monitoring.
payload	Data yang dikirim ke client dalam format JSON.
status	Status pengiriman: <code>PENDING</code> , <code>PROCESSING</code> , <code>SENT</code> , <code>FAILED</code> , <code>DEAD_LETTER</code> .
retry_count	Jumlah percobaan kirim ulang.
max_retry	Batas maksimal retry otomatis.
next_retry_at	Waktu retry berikutnya.
last_error	Error terakhir dari proses pengiriman.
sent_at	Waktu data berhasil dikirim.
created_at	Waktu data dibuat.
updated_at	Waktu data diperbarui.

7. Catatan Implementasi

- Gunakan transaksi saat mengambil dan mengubah status data menjadi `PROCESSING`.
- Gunakan HTTP client dengan timeout.
- Simpan error secukupnya di `last_error`, tanpa menyimpan data sensitif.
- Gunakan idempotency key jika endpoint client mendukungnya agar retry tidak membuat data duplikat di sisi client.
- Pantau jumlah data `FAILED` dan `DEAD_LETTER` untuk kebutuhan investigasi.